Training a Perceptron with the Perceptron Trick

Contents

1. Overview 2
2. Prerequisites 3
3. The trick in one picture 3
4. The trick in perceptron terms 5
5. The loop on real data 6
6. When to stop 7
7. Loops and epochs 8
8. Why it is only a trick 9
9. Summary 10
10. Sources 10
11. Key terms 11

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Perceptron trick.

1. Overview

Key point: Training a perceptron means finding its weights and bias (the numbers that define its line). The perceptron trick does it by starting from any line and pulling it towards every misclassified point it picks, until a **stopping rule** (G-1894) ends the loop.

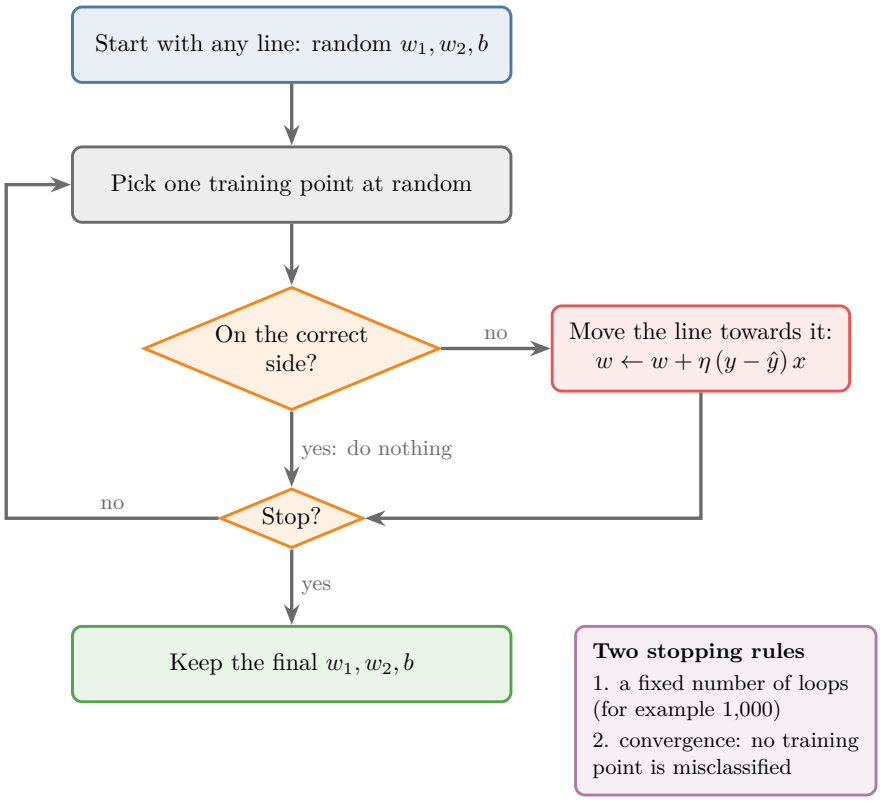


Figure 1: The perceptron trick as a training loop, with its two stopping rules

Earlier we saw how a trained perceptron predicts. This Note covers how it gets its numbers: the **perceptron trick** (G-1485), the simplest training method. Figure 1 shows the whole loop.

The trick itself is taught in full in two earlier Notes, and we do not repeat it here:

- the **perceptron trick** (G-1485): positive and negative sides of a line, how A, B and C move it, the add-or-subtract update, the **learning rate** (G-1068) (its Figure 7 animates

the line swinging towards a misclassified point in small steps), and the single rule $w \leftarrow w + \eta(y - \hat{y})x$;
- the code of the trick: the code, an animation of the line moving update by update (its Figure 3), and why the final line depends on the random order.

This Note has these parts:

1. a one-picture recap of the trick (section 3);
2. the trick in the perceptron's own terms (section 4);
3. the loop running on real data (section 5);
4. when to stop (section 6);
5. loops against **epochs** (G-696) (section 7);
6. why the trick is not enough (section 8).

2. Prerequisites

- The parts of a perceptron: weights, bias, summation and step activation.
- The **perceptron trick** (G-1485) and its code.

3. The trick in one picture

> **Key point:** C slides the line, A and B turn it. Subtracting a wrongly placed negative point from (A, B, C) turns and slides the line until the point is on the correct side.

The trick works like a conversation between the line and the points. We start with any line and ask one randomly picked point: "are you on the correct side?" A point on the correct side answers "leave the line where it is". A point on the wrong side answers "come towards me", and the line moves towards that point. Repeating the question many times brings the line to a place where no point complains.

The data must be **linearly separable** (G-1103): a straight line can split the two classes. Otherwise some point always complains.

To move the line we change the three numbers of its equation $Ax + By + C = 0$. Figure 2 shows what each number does, and then one full move.

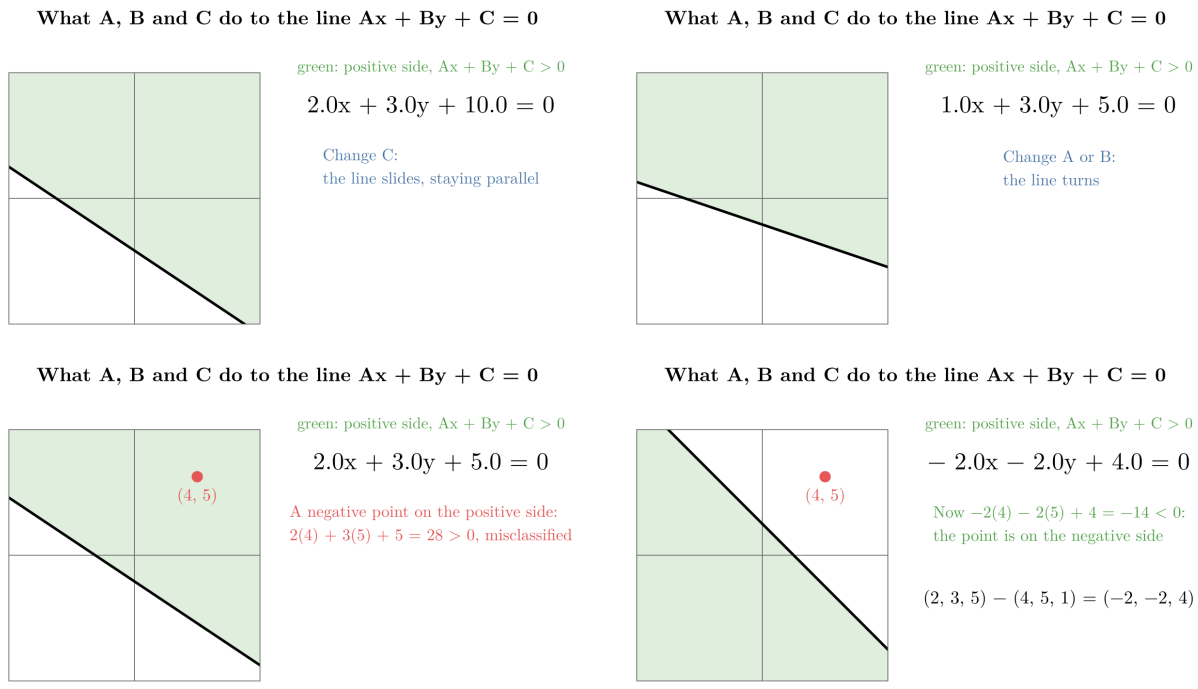


Figure 2: The line $2x + 3y + 5 = 0$. First C changes, then A, then B. Then the negative point $(4, 5)$, which sits on the positive (green) side, is subtracted from the coefficients

In Figure 2, watch three things in order:

1. **Changing** C slides the line while keeping it parallel to itself.
2. **Changing** A **or** B turns the line.
3. **One update.** The point $(4, 5)$ belongs to the negative class. Put it in the line $2x + 3y + 5$:

$$2(4) + 3(5) + 5 = 8 + 15 + 5$$
$$2(4) + 3(5) + 5 = 28 > 0$$

So it sits on the positive side: it is misclassified. We append a 1 to the point and subtract it from the coefficients:

$$(2,\ 3,\ 5) - (4,\ 5,\ 1) = (-2,\ -2,\ 4)$$

The new line is $-2x - 2y + 4 = 0$. Put the point in it:

$$-2(4) - 2(5) + 4 = -8 - 10 + 4$$
$$-2(4) - 2(5) + 4 = -14 < 0$$

The point is now on the negative side, as it should be.

The update changes all three numbers at once, so the line turns and slides in one move. For a positive point on the negative side we add instead of subtracting. In practice we do not subtract the whole point: we multiply it by a small **learning rate** (G-1068), such as 0.01, so the line moves in small steps. The **perceptron trick** (G-1485) works through both cases, and the learning rate is taught there too.

4. The trick in perceptron terms

> **Key point:** The line Ax + By + C = 0 of the trick is the perceptron's z = 0. A and B are the weights, C is the bias, and the column of 1s is the bias input.

The **perceptron trick** (G-1485) works on the line $Ax_1 + Bx_2 + C = 0$. The perceptron's **decision boundary** (G-555), the line that separates the points it calls positive from those it calls negative, is $z = w_1x_1 + w_2x_2 + b = 0$. They are the same line with different names:

Perceptron trick	Perceptron	Role
A (w_1)	weight w_1	how much x_1 counts
B (w_2)	weight w_2	how much x_2 counts
C (w_0)	bias b	shifts the line
column $x_0 = 1$	the constant input 1	carries the bias

So the column of 1s added in the trick's code is exactly the extra input "1" of the perceptron diagram (Figure 1 of the parts of a perceptron), and the 1 we appended to the point $(4, 5)$ in section 3 is that same input.

The two cases, add or subtract, fold into one rule. With the label y and the prediction $\hat{y}$ both 0 or 1:

$$w \leftarrow w + \eta\,(y - \hat{y})\,x$$

Here $w = (w_0, w_1, w_2)$ holds the bias and the two weights, $x = (1, x_1, x_2)$ is the point with its 1, and η is the **learning rate** (G-1068). The four cases of $(y, \hat{y})$ show that the rule does the right thing every time:

y	$\hat{y}$	$y - \hat{y}$	What the rule does
1	1	0	nothing: the point is correct
0	0	0	nothing: the point is correct
1	0	+1	adds ηx: a positive point on the negative side
0	1	−1	subtracts ηx: a negative point on the positive side

The rule changes both weights and the bias in one step.

Training and prediction then split cleanly:

1. **Training:** run the loop of Figure 1 on the labelled data. Its only output is the three numbers w_1, w_2 and b.
2. **Prediction:** freeze them, compute z for a new point and apply the step function.

5. The loop on real data

> **Key point:** Each loop picks one point. A correctly classified pick leaves the line still; a misclassified pick moves it. Most picks leave the line still.

Figure 3 runs the rule of section 4 on real flowers: 50 setosa (label 0) and 50 versicolor (label 1) from the Iris dataset, described by petal length (x-axis) and petal width (y-axis), with **learning rate** (G-1068) 0.1. Each dot is one flower (blue setosa, orange versicolor) and the black line is the perceptron's decision boundary.

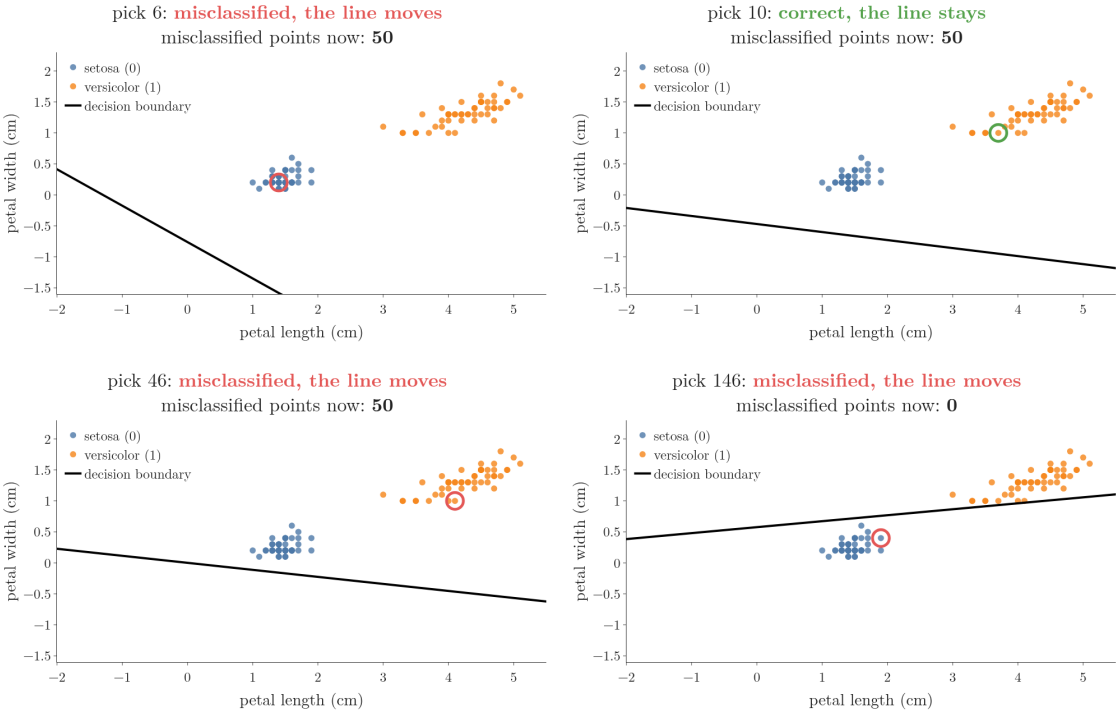


Figure 3: The perceptron trick pick by pick on Iris setosa and versicolor. The ring marks the picked flower: green when it is correctly classified, red when it is misclassified

In Figure 3, watch the ring and the line together:

- **Green ring:** the picked flower is on the correct side, $y - \hat{y} = 0$, and the line does not move.
- **Red ring:** the picked flower is on the wrong side, and the line jumps towards it.

Of the 146 picks in this run, only 26 move the line; the other 120 leave it still. The counter in the title shows the number of misclassified flowers after each pick. The count starts at 50, because the starting line puts every flower on the positive side, and reaches 0 at pick 146.

6. When to stop

> **Key point:** Either run a fixed number of loops, or stop at convergence: the moment no training point is misclassified.

The loop in Figure 1 needs a **stopping rule** (G-1894) to end it. Two are common:

1. **A fixed number of loops**, for example 1,000. If the line is not good yet, run more, such as 10,000.
2. **Convergence:** after every update, count the misclassified training points. Stop as soon as the count is 0, since no point can move the line any more.

Rule 2 only ever stops on **linearly separable** (G-1103) data: if no line separates the classes, some point is always misclassified. Data that no line separates is why the two rules are often combined: stop at convergence, or after the maximum number of loops, whichever comes first.

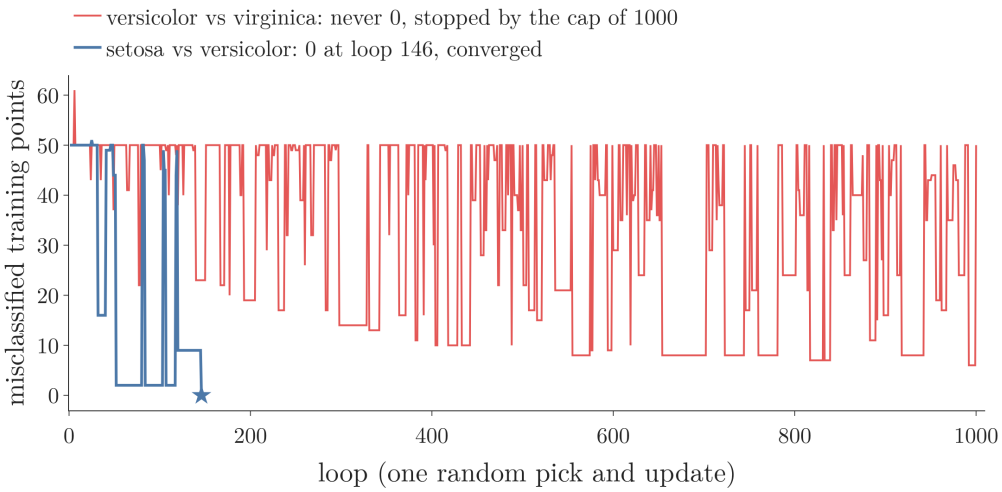


Figure 4: The perceptron trick on two pairs of Iris species (petal length and width, 100 flowers each, learning rate 0.1), counting the misclassified training points after every loop

Figure 4 runs the combined rule on real data. Watch the blue line: setosa and versicolor can be split by a line, so the count hits 0 at loop 146 and convergence stops training (the run of Figure 3). The red line, versicolor against virginica, never reaches 0 (its best is 6), so only the cap of 1,000 loops ends it. Six other random orders give the same picture.

> **Extra:** scikit-learn's `Perceptron` uses a similar combination, with a looser early stop. `max_iter` caps the number of **epochs** (G-696) (default 1,000). With `tol` set (default 10^{-3}), training also stops once the loss has failed to improve by at least `tol` for `n_iter_no_change` epochs in a row (default 5) (scikit-learn docs, `Perceptron`). This stop can fire before every point is classified correctly: in the perceptron in scikit-learn, it ended training after 7 epochs at 75% training accuracy.

7. Loops and epochs

> **Key point:** One random pick is one update, not one epoch. An epoch is one pass over the whole training set, so with 100 points, 1,000 single picks are about 10 epochs.

Each loop of the trick looks at one randomly chosen point. The number of loops is sometimes loosely called the number of "epochs", but that is not the standard meaning.

An **epoch** (G-696) is one full pass over the training set. In the trick, one pass means as many picks as there are points:

1. **In words:** divide the number of single-point updates by the number of training points.
2. **Formula:**
$$\text{epochs} = \frac{\text{loops}}{\text{points}}$$
Here "loops" is the number of single-point updates and "points" is the number of training points.
3. **Example:** 1,000 loops on 100 points:
$$\frac{1000}{100} = 10 \text{ epochs}$$

Because the picks are random, a stretch of 100 picks does not visit every point once. A given point is missed by one pick with probability $1 - 1/100$, so it is missed by all 100 picks with the probability below, step by step:

$$1 - \tfrac{1}{100} = 0.99 \quad \text{(missed by one pick)}$$
$$0.99^{100} \approx 0.37 \quad \text{(missed by all 100 picks)}$$

In each stretch, about a third of the points are not seen, while others are seen twice or more.

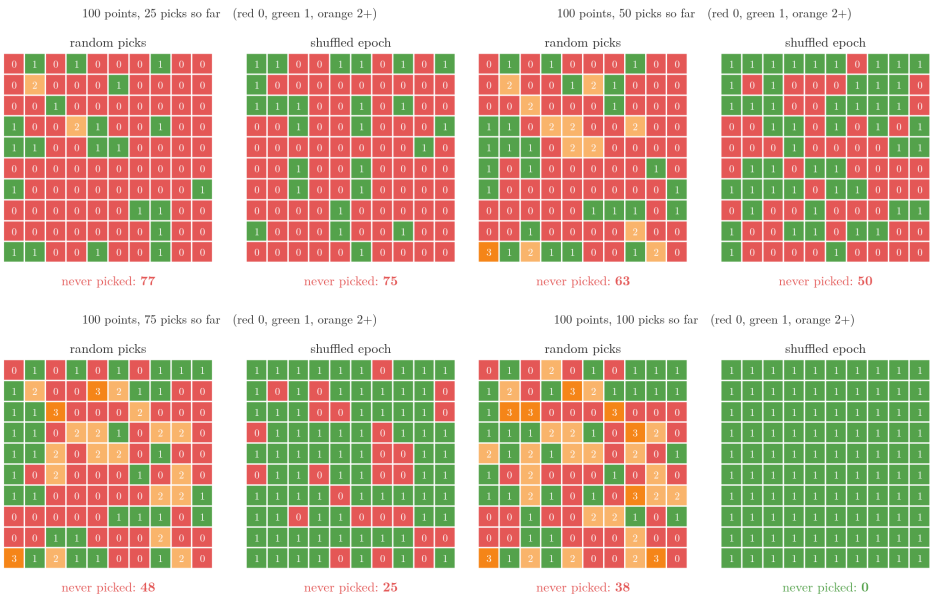


Figure 5: Each cell is one of 100 training points, and its number counts how often it has been picked. Left: 100 random picks, as in the perceptron trick; 38 points are never picked while others are picked 2 or 3 times. Right: one shuffled epoch; every point is picked exactly once

In Figure 5, watch the red cells: on the right they all turn green by pick 100; on the left 38 stay red. Over 2,000 runs the average is 36.6 unseen points, matching $(1 - 1/100)^{100} \approx 0.37$ (the figure's script, `images/random_picks.py`, runs the check). Visiting the points in a shuffled order, each once per epoch, avoids this; that is how **stochastic gradient descent** (G-1892) (gradient descent that updates after one point or a small batch) goes through the data.

8. Why it is only a trick

> **Key point:** The trick finds a separating line, but cannot say how good that line is, and different random orders give different lines.

The **perceptron trick** (G-1485) is simple and usually works. But on the same data, different random orders give different final lines, some hugging one class (see the weakness of the trick). Nothing in the trick measures which line is better. The trick is like a student who stops revising the moment every practice question is right: they pass, but they never learn their score, so they cannot tell a narrow pass from a safe one.

The fix is a loss function: a number that scores every possible line, so training can look for the best one. The **perceptron loss** (G-1484) builds it.

9. Summary

Idea	Meaning	Why
Training	Finding w_1, w_2 and b from labelled data	these three numbers fully define the line
Perceptron trick	Pick a random point; if it is misclassified, pull the line towards it	only a point on the wrong side has a reason to move the line
C; A and B	C slides the line; A and B turn it	one update changes all three, so the line turns and slides in one move
Update	$w \leftarrow w + \eta(y - \hat{y})x$, with $x_0 = 1$ for the bias	one rule covers all four cases: add, subtract or leave the line
Stop: fixed loops	Run, for example, 1,000 picks	it always ends, even on data no line separates
Stop: convergence	Stop when no training point is misclassified	no point is left that could move the line
Epoch	One pass over the whole training set, not one pick	one random pick sees only one point, so loops and epochs count different things

- The trick's A, B, C are the perceptron's w_1, w_2, b, so the trick trains the perceptron's own **decision boundary** (G-555).
- A correctly classified pick leaves the line still, because $y - \hat{y}$ is 0 for it; only misclassified picks move it.
- Convergence is only reached on **linearly separable** (G-1103) data, so a maximum number of loops is kept as a backstop.
- 1,000 random picks on 100 points are about 10 **epochs** (G-696), because an epoch is one pass over all the training points.
- The trick cannot score a line; a loss function can, so different random orders give different lines and only a loss can say which one is better.
- So the perceptron trick trains a perceptron by pulling its line towards misclassified points until a stopping rule ends the loop, and its missing score is what the **perceptron loss** (G-1484) supplies.

10. Sources

Built from

- CampusX, "Perceptron Trick | How to train a Perceptron | Perceptron Part 2 | Deep Learning Full Course", YouTube, https://www.youtube.com/watch?v=Lu2bruOHN6g

Other references

- scikit-learn documentation, `sklearn.linear_model.Perceptron`.

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Stopping rule (G-1894)	The condition that ends a training loop: a fixed number of loops, or convergence.
Epoch (G-696)	One full update of the parameters using the whole training set.
Training (a perceptron) (G-2000)	Finding its weights and bias from labelled data.
Perceptron trick (G-1485)	Moving a line towards each misclassified point until the classes are separated.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Linearly separable (G-1103)	Data whose classes a straight line, plane or hyperplane can split.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.
Perceptron loss (G-1484)	A loss for the perceptron that is 0 for a point on its correct side and $\lvert f(x)\rvert$ for a point on the wrong side, so worse mistakes cost more: $\max(0, -yf(x))$ per point, labels ± 1. Minimising it trains the line.